Homework 1

CSE 2046/2246 Analysis of Algorithms, Spring 2026

Due 05.05.2026, 23:59

Design and conduct an experiment to compare the following three string matching algorithms:

1. Brute-force string matching
2. Horspool's algorithm
3. Boyer-Moore algorithm

You should implement all three algorithms in any programming language.

The input will be:

- a long HTML file (longer than 1 MB), and
- a pattern.

Each algorithm must:

- search the given pattern in the HTML file,
- find **all occurrences** of the pattern,
- report the **number of occurrences**,
- report the **number of character comparisons**,
- report the **running time** in milliseconds,
- generate an output HTML file in which each occurrence is highlighted using the <mark></mark> tag.

For Boyer-Moore, you should implement the version discussed in class, including both the **bad symbol rule**, and the **good suffix rule.** For each pattern, your program should also print:

- the **bad symbol table**, and
- the **good suffix table**.

Your report must include detailed comments and discussion on the observed results.

Important Clarifications

1. The algorithms should search on the **raw HTML text directly**. You are not required to parse the HTML DOM or search only the visible text.
2. All occurrences of the pattern should be found. **Overlapping matches should also be counted**, if they exist.
3. The reported running time should correspond to the **search process only**. File reading, output file generation, and writing the highlighted HTML file should **not** be included in the reported search time. However, your report should explain your implementation

decisions regarding file reading, memory usage, and output generation.

4. The algorithms should be implemented **exactly as described in the lecture**.

Input Types

You should work on two types of HTML files:

Type 1: HTML files with English text

You may download long HTML files from the Internet. Patterns should be meaningful English words or phrases.

Type 2: HTML files containing random bit-strings

You may generate these files yourself. Patterns should be random bit-strings of various lengths.

For **each type**, provide **at least 3 different input samples**.

Thus, you should test your algorithms on **at least 6 different HTML files in total**.

Step 1: Designing the Experiment

This step includes:

(a) Deciding on / generating long HTML files

You should clearly describe:

- how the files were selected or generated,
- the source / URL of each file if applicable,
- the length of each file,
- any other relevant properties of the file content.

(b) Deciding on patterns

You should clearly describe:

- how the patterns were selected,
- the length of each pattern,
- whether the pattern contains repeated characters,
- whether the pattern occurs frequently or rarely in the text,
- why these patterns were chosen.

Your pattern selection should be designed carefully so that the effects of factors such as:

- pattern length,
- repeated characters,

- first/last character properties,
- frequent vs. infrequent occurrences,
- structured vs. random text

can be observed in the experiments.

To ensure meaningful comparison, you should include patterns with **different lengths and characteristics**. For example, your experiments should include a mixture of:

- short patterns,
- medium-length patterns,
- long patterns,
- patterns with repeated characters,
- patterns with few or no repeated characters,
- patterns with many matches,
- patterns with very few matches.

Step 2: Coding and Running

All three algorithms should be implemented and run on the selected HTML files and patterns.

Your programs should produce the following outputs:

(a) Preprocessing tables

For each pattern, print:

- the bad symbol / bad character table,
- the good suffix table.

(b) Highlighted output file

Generate an output HTML file in which all matched occurrences are highlighted using <mark></mark>.

(c) Performance results

For each algorithm and each test case, report:

- number of occurrences,
- number of character comparisons,
- running time in milliseconds.

Additional Correctness Test Cases

In addition to the long HTML files and selected patterns, you should also run your code on the

following small test cases and provide the outputs. These test cases are intended to demonstrate that your implementation works correctly for basic scenarios.

Test Case 1: Single Match

Text:
<HTML><BODY>WHICH_FINALLY_HALTS. _ _ AT_THAT POINT
</BODY></HTML>

Pattern:
AT_THAT

Test Case 2: Multiple Matches

Text:
<HTML><BODY>ABC_AT_THAT_XYZ_AT_THAT_END</BODY></HTML>

Pattern:
AT_THAT

Test Case 3: Overlapping Matches

Text:
<HTML><BODY>AAAAAA</BODY></HTML>

Pattern:
AAA

Test Case 4: No Match

Text:
<HTML><BODY>THIS_IS_A_TEST</BODY></HTML>

Pattern:
XYZ

For each of the above test cases, you should report:

- the number of occurrences,
- the number of character comparisons,
- the running time,
- and the highlighted output HTML file.

If your code does not work correctly for these test cases, or if you do not provide the corresponding outputs, your code will be considered as **not working correctly**.

Step 3: Illustrating and Analyzing Results

This step includes:

(a) Presenting results

Present your results using appropriate:

- tables,
- plots,
- and/or charts.

(b) Comparing the algorithms

Compare the performance of all three algorithms for different:

- file types,
- file sizes,
- pattern types,
- pattern lengths,
- match frequencies.

(c) Discussing findings

Provide detailed comments on your findings.

Your discussion should address questions such as:

- Which algorithm performs best under which conditions?
- How does pattern length affect performance?
- How do repeated characters in the pattern affect performance?
- How does random text differ from English text?
- Are the experimental results consistent with the theoretical expectations?
- If not, what implementation or input-related reasons may explain the differences?

Report Requirements

Your detailed report should include at least the following sections:

1. **Introduction**
2. **Algorithms Implemented**
3. **Experimental Setup**
    - input files
    - pattern selection
    - platform / machine information
4. **Implementation Details**
    - file reading

    - memory usage
    - highlighting method
    - counting character comparisons
    - timing methodology
5. **Results**
6. **Discussion**
7. **Conclusion**
8. **Division of Labor**

Notes

Note 1: You may do this homework in groups of three.

Note 2: Some old browsers may not support the <mark> tag. Please use a browser that supports this tag for displaying the output HTML file.

Note 3: Please submit your commented source code, input files, output files, and detailed report in a single zip file via Google Classroom. The name of the zip file should include the names and surnames of all group members. It is enough if one group member submits the homework.

Note 4: Clearly describe the division of labor among group members.

Note 5: Although correct implementation is required, your grade will depend largely on the quality of your analysis, discussion, and presentation in the report.